



CalendarMundial 2026

Documentación Funcional y Técnica

App iOS · Swift 5.9 · SwiftUI · Xcode 16

Autor: Francisco Cornellana Castells

Repositorio: github.com/cornellana/calendar-mundial-2026

Julio 2026

1. Resumen de la aplicación

CalendarMundial 2026 es una app nativa iOS/iPadOS que ofrece el calendario completo de la Copa del Mundo FIFA 2026 con resultados en tiempo real, alineaciones detalladas, filtros avanzados y clasificaciones de grupo. No requiere cuenta de usuario ni backend propio: los datos se sirven desde un repositorio público de GitHub actualizado automáticamente cada 15 minutos mediante un workflow de GitHub Actions que consulta la API pública de ESPN.

Plataforma	iOS 17+ / iPadOS 17+
Idioma UI	Español (horarios en zona horaria de Madrid, CEST UTC+2)
Distribución	TestFlight / App Store · Bundle ID: com.cornellana.CalendarMundial
Equipo	Francisco Cornellana Castells · Apple Team ID: TJ6V4QM3GB
Repositorio	https://github.com/cornellana/calendar-mundial-2026
Torneo	FIFA World Cup 2026 · 11 jun – 19 jul 2026 · Sede: EE.UU., Canadá y México

2. Funcionalidades principales

Calendario y resultados

- Listado cronológico de todas las jornadas agrupadas por día.
- Badge de fase por jornada: Grupos · 1/16 · 1/8 · Cuartos · Semis · Final.
- Indicador **HOY** (pin dorado) anclado al día con partidos más próximo a la fecha actual; cuando no hay partido exactamente hoy, muestra **PRÓXIMO** en el siguiente día con partido.
- Scroll automático al partido del día al abrir la app o al volver al primer plano.
- Resultado en pantalla y opacidad reducida para partidos ya disputados.
- Pull-to-refresh manual para forzar la descarga del JSON remoto.

Detalle de partido

- Hoja modal con alineaciones titulares (4-4-2, 4-3-3, etc.) de ambos equipos.
- Eventos por jugador: goles , penaltis, goles en propia puerta, amarillas , rojas , sustituciones.
- Minuto exacto de cada evento, incluido tiempo añadido ("45+3").
- Posición en español (POR · DEF · MED · DEL) y dorsal del jugador.
- Estadísticas de posesión y tiros (cuando ESPN las publica).

Filtros y búsqueda

- Filtro por fase del torneo (chips deslizantes).
- Filtro por grupo (A–L): muestra clasificación del grupo en tiempo real.
- Filtro por país anfitrión (México · Canadá · EE.UU.).
- Filtro por estadio.
- Búsqueda por nombre de equipo (case-insensitive, localizada).

Clasificación de grupos

- Al seleccionar un grupo, aparece una tabla de clasificación con PJ · G · E · P · GF · GC · DG · Pts.
- Calculada en tiempo real a partir de los resultados registrados en el JSON remoto.

Goleadores

- Hoja modal con el ranking de goleadores del torneo ordenado por goles.
- Calculado a partir de los eventos de tipo *goal* y *penalty* de todas las alineaciones.

Indicadores de TV para España

- **La 1 + DAZN** (punto azul): emisión gratuita en abierto por TVE.
- **Solo DAZN** (punto dorado): emisión de pago.
- Partidos de España resaltados con fondo verde y texto en verde claro.

3. Arquitectura técnica

La app sigue el patrón **MVVM** con el Observation framework de Swift 5.9. No usa Combine ni UIKit. Toda la capa de datos vive en **MatchStore**, un actor observable marcado con **@MainActor** que garantiza las actualizaciones de UI en el hilo principal.

Carga de datos (cascada de tres fuentes)

Prioridad	Fuente	Descripción
1 – Caché	UserDefaults (clave <i>mundial2026_cache_v2</i>)	Snapshot JSON serializado del último refresh exitoso. Disponible offline e instantáneo al arrancar.
2 – Seed	MundialData.matchDays (bundle)	Datos compilados en el binario. Se usa solo si no hay caché. Siempre coherentes con la versión publicada.
3 – Remoto	GitHub Raw CDN → JSONDecoder	JSON descargado al activar la app (<code>scenePhase == .active</code>). Sobrescribe caché y seed.

Flujo de actualización automática

- GitHub Actions cron cada **15 minutos** ejecuta **scripts/update_mundial.py**.
- El script consulta la API pública de ESPN (*site.api.espn.com/apis/site/v2/sports/soccer/fifa.world*).
- Para partidos finalizados descarga detalle completo: alineaciones, eventos, estadísticas.
- Aplica mapeo de nombres ESPN → español y escribe **data/mundial2026.json**.
- Si hay cambios, el bot hace commit (*github-actions[bot]*) y push a *main*.
- La app descarga el JSON con cache-busting (*?t=timestamp*) y política *reloadIgnoringLocalAndRemoteCacheData*.

Capas del código fuente

Archivo	Responsabilidad
Models.swift	Tipos de dominio: Match, MatchDay, Phase, LineupPlayer, MatchEvent, MatchSnapshot...
MatchesData.swift	Seed data estático con el calendario completo. Sirve de fallback offline.
MatchStore.swift	ViewModel @Observable: carga, caché y refresh remoto. @MainActor.
ContentView.swift	Vista raíz: filtros, lista de jornadas, scroll automático, hoja de detalle.
MatchDetailSheet.swift	Hoja modal con alineaciones y eventos de un partido concreto.
MatchDetailsData.swift	Datos de detalle compilados en el bundle (se sobrescriben con los del JSON remoto).

Archivo	Responsabilidad
TopScorersSheet.swift	Hoja modal con el ranking de goleadores calculado en tiempo real.
Assets.xcassets	Icono de la app y paleta de colores.
Localizable.xcstrings	Strings localizables (formato String Catalog de Xcode 15+).

4. Modelos de datos

Match — partido individual

Campo	Tipo	Descripción
time	String	Hora de inicio en Madrid (CEST), p. ej. "21:00".
home	String	Nombre del equipo local.
away	String	Nombre del equipo visitante.
group	String	Identificador: letra de grupo (A–L), "1/16", "CF", "SF", "FINAL".
tv	TVChannel	Canal de emisión: .both (La 1 + DAZN) o .dazn.
done	Bool	true cuando el partido ha terminado.
result	String?	Marcador final, p. ej. "2-1" o "1-1 p." (penaltis).
esp	Bool	true si juega la selección española → fondo verde.
details	MatchDetails?	Alineaciones y eventos; nil hasta que ESPN publica el detalle.
stadium	String?	Nombre del estadio.
hostCountry	String?	País anfitrión (México/Canadá/EE.UU.) para filtro geográfico.

MatchDay — jornada

Campo	Tipo	Descripción
date	String	Fecha ISO "YYYY-MM-DD" en zona horaria de Madrid.
phase	Phase	Fase del torneo (.grupos, .dieciseisavos, .octavos, .cuartos, .semis, .final).
games	[Match]	Partidos disputados ese día.

MatchEvent — evento en partido

Campo	Tipo	Descripción
type	MatchEventType	.goal · .penalty · .ownGoal · .yellow · .red · .subIn · .subOut
minute	Int	Mínuto reglamentario (1–120).
extraTime	Int?	Tiempo añadido (p. ej. 3 para "45+3"). nil si no aplica.

MatchSnapshot — sobre el JSON remoto

El JSON que descarga la app (y que guarda en UserDefaults) se codifica como **MatchSnapshot**, un wrapper que añade la fecha de última actualización (*lastUpdated: Date*) al array de *matchDays*. El decoder usa **.iso8601** para parsear fechas.

5. Integración con la API de ESPN

La API de ESPN es pública, sin autenticación, sin API key y sin coste. Devuelve JSON con datos de competiciones de fútbol. Se consulta solo desde GitHub Actions (servidor), nunca desde el dispositivo del usuario.

Endpoints utilizados

Scoreboard (partidos del día)

GET `site.api.espn.com/apis/site/v2/sports/soccer/fifa.world/scoreboard?dates=YYYYMMDD`

Detalle de un partido (alineaciones + eventos)

GET `site.api.espn.com/apis/site/v2/sports/soccer/fifa.world/summary?event={espn_id}`

Campos clave del scoreboard

Campo ESPN	Uso
<code>competitions[0].competitors[].homeAway</code>	Identifica local ("home") y visitante ("away").
<code>competitions[0].competitors[].score</code>	Goles marcados (string).
<code>competitions[0].status.type.completed</code>	Bool: true = partido terminado. Se usa para activar la descarga del detalle.
<code>competitions[0].status.type.name</code>	STATUS_FULL_TIME, STATUS_SCHEDULED, etc. Solo informativo.
<code>competitions[0].date</code>	Fecha/hora UTC en formato ISO 8601.
<code>competitions[0].venue.fullName</code>	Nombre completo del estadio.

Parsing de jugadores y eventos

En el endpoint *summary*, cada jugador aparece en `rosters[].athletes[]` con un array `plays[]` que registra sus acciones. La lógica de extracción:

- Solo se procesa *scoringPlay: true* para filtrar jugadas sin relevancia.
- **didScore: true** → gol real del jugador.
- **didAssist: true** (sin didScore) → asistencia, se ignora para el marcador.
- El campo *clock.displayValue* puede ser "45+3" → se parsea con regex para separar minuto y tiempo extra.
- Posición: ESPN usa "GK/D/M/F" → mapeado a "POR/DEF/MED/DEL".

Detección de goles en propia puerta y penaltis

El campo *shootoutPlay* indica penalti en la tanda. Los goles en propia puerta se detectan comparando el nombre del equipo que anota con el equipo al que pertenece el jugador. Los penaltis durante el partido (no tanda) se identifican por *penaltyKick: true*.

6. Estructura del repositorio

```
calendar-mundial-2026/
├── CalendarMundial/ # Xcode project
│   ├── CalendarMundial/ # Sources del target
│   │   ├── Models.swift
│   │   ├── MatchesData.swift # Seed data
│   │   ├── MatchStore.swift # @Observable ViewModel
│   │   ├── ContentView.swift # Vista raíz
│   │   ├── MatchDetailSheet.swift
│   │   ├── TopScorersSheet.swift
│   │   ├── Assets.xcassets / Localizable.xcstrings
│   │   └── CalendarMundial.xcodeproj # Generado con XcodeGen
│   └── data/
│       ├── mundial2026.json # JSON actualizado por el cron
│       ├── scripts/
│       │   ├── update_mundial.py # Fetcher ESPN → JSON
│       │   └── .github/workflows/
│       ├── update-mundial.yml # Cron cada 15 min
│       └── project.yml # Spec XcodeGen
```

GitHub Actions workflow

Parámetro	Valor
Trigger	schedule: */15 * * * * (cada 15 min) + workflow_dispatch
Runner	ubuntu-latest
Timeout	5 minutos
Permisos	contents: write (para hacer commit del JSON)
Variable secreta	Ninguna — la API de ESPN es pública sin autenticación
Force refresh	Input booleano: reprocesa todos los partidos finalizados (útil tras bugs)

URL de consumo desde la app

```
https://raw.githubusercontent.com/cornellana/calendar-mundial-2026/refs/heads/main/data/mundial2026.json
```

Se añade un parámetro `?t={unix_timestamp}` como cache-buster para evitar que el CDN de GitHub Raw (caché de ~5 min) sirva datos obsoletos. Adicionalmente el `URLRequest` usa política `reloadIgnoringLocalAndRemoteCacheData` y cabecera `Cache-Control: no-cache`.

7. Diseño de interfaz

Paleta de colores

Token	Hex	Uso
Navy dark	#0A0F1E	Fondo principal de toda la app.
Navy medium	#0D1F3C	Fondo de filas y tarjetas.
Navy light	#1E3A5F	Bordes, separadores, chips inactivos.
Gold	#C8A84B	Acento principal: chips activos, pin HOY, resultados.
Gold light	#F0D070	Texto de partidos de la Final.
Blue light	#E0EAFF	Texto de cuerpo (equipos, fechas futuras).
Slate	#4A6080	Texto secundario y fechas pasadas.
Green	#90E890	Texto de partidos de España.
Blue ESPN	#2196F3	Punto del badge La 1 + DAZN.

Componentes principales

- **DaySectionView**: cabecera con fecha, pin HOY/PRÓXIMO y badge de fase. Lista de MatchRow debajo.
- **MatchRow**: hora · equipos · resultado/grupo · badge TV · chevron de acceso al detalle.
- **PhaseChip**: chip pill para filtros (mínimo 44×36 pt de tap target).
- **MatchDetailSheet**: hoja modal con alineaciones en grid de dos columnas (local | visitante).
- **TopScorersSheet**: lista ordenada por goles con nombre, equipo y contador.
- **GroupStandingsView**: tabla de clasificación inline cuando el filtro de grupo está activo.

Scroll automático

Al activarse la app (*scenePhase* == *.active*), tras un sleep de 350 ms para que SwiftUI termine el layout, se ejecuta `proxy.scrollTo(scrollTargetDate, anchor: .top)`. El `scrollTargetDate` es el primer día con partidos cuya fecha sea $\geq$ today; si el torneo ya terminó, el último día. La misma lógica se aplica al cambiar el filtro de fase o al cerrar la hoja de goleadores.

8. Gestión de zona horaria

Todos los horarios de la app se expresan en hora de Madrid (Europa/Madrid, CEST UTC+2 en verano). El campo *time* de cada partido es un string "HH:mm" ya convertido. La función **MundialData.todayString** computa la fecha actual en la zona horaria de Madrid para que el indicador HOY sea correcto independientemente de dónde se encuentre el usuario.

```
static var todayString: String {  
    let tz = TimeZone(identifier: "Europe/Madrid") ?? .gmt  
    return Date().formatted(  
        Date.ISO8601FormatStyle(timeZone: tz).year().month().day()  
    )  
}
```

La propiedad es un **var computado** (no **let**) en ContentView para que se recalcule en cada render, evitando que el valor quede obsoleto si la app cruza medianoche sin reiniciarse.

9. Caché local y versionado

El snapshot JSON se persiste en **UserDefaults** bajo la clave `mundial2026_cache_vN`. Al bumpar N se invalida la caché de todos los usuarios sin necesidad de publicar una nueva versión de la app: en el siguiente arranque, la app no encuentra la clave antigua, carga el seed actualizado y descarga el JSON remoto.

Versión clave	Motivo del cambio
<code>mundial2026_cache_v1</code>	Versión inicial.
<code>mundial2026_cache_v2</code>	Invalidación forzada para cargar equipos del 3P y la Final tras actualización de julio 2026.

10. Requisitos y compilación

Requisito	Versión / Detalle
Xcode	16.0 o superior
Swift	5.9+
iOS deployment	17.0+
XcodeGen	2.x (brew install xcodegen) — para regenerar el <code>.xcodeproj</code>
Firma	Automática · Team: Francisco Cornellana Castells (TJ6V4QM3GB)
GitHub CLI (gh)	Para gestión de repo y secrets (opcional en desarrollo local)
Python 3.11	Solo para el script de actualización en GitHub Actions
requests (pip)	Única dependencia Python del script de actualización

Regenerar el proyecto tras añadir/mover archivos:

```
xcodegen generate
xcodebuild build -project CalendarMundial.xcodeproj \
-scheme CalendarMundial \
-destination "generic/platform=iOS Simulator"
```